



Software Engineering Department
Braude College of Engineering, Karmiel

Capstone Project - Phase A (61998)

RightRent

An AI-Based Web Application for Analyzing Residential Rental Contracts
under the Israeli Fair Rental Law

Project Name: RightRent

Code: 26-2-D-6

Submitted by:

Ashraf Atshy
Kian Gadban

Advisor: Mr. Eyal Sade

GitHub Repository: <https://github.com/ashrafatshy7/RightRent>

Table of Contents

Abstract	3
Chapter 1: Description of Needs and Project Goals (Introduction).....	4
1.1 Background and Current Situation.....	4
1.2 Problem Definition and Proposed Solution.....	4
1.3 Project Goals	5
1.4 Structure of the Book and Literature Review.....	5
Chapter 2: Problem Analysis and Solution Formation	5
2.1 Analysis of the Current Situation: The Israeli Rental Market and Contracts of Adhesion	5
2.2 Problem Analysis: Information Asymmetry, Legal Readability, and Conflict with the Law.....	6
2.3 Solution Formation: Semantic Analysis, Personalization, and Technological Considerations	6
Chapter 3: Research and Engineering Process.....	7
3.1 Review of Research and Engineering Challenges.....	7
3.2 Detailed Requirements	8
3.2.1 Functional Requirements	8
3.2.2 Non-Functional Requirements.....	9
3.3 Research Approaches Considered.....	9
3.4 Technology Selection.....	10
3.5 Architecture and Analysis Process.....	10
3.6 The Negotiation Agent	11
3.7 Work Plan and Timeline	12
3.8 Constraints, Risks, and Mitigation Strategies	12
3.9 Success Metrics	13
Chapter 4: Work Artifacts.....	13
4.1 Overview of Project Deliverables	13
4.2 Activity Diagram.....	14
4.2.1 Scenario 1: Contract Upload and AI Lease Analysis	14
4.2.2 Scenario 2: Negotiation Agent and WhatsApp Message Preparation	14
4.2.3 Scenario 3: API-Based Law Knowledge Base Synchronization	15
4.3 System Architecture and Software Structure	15
4.4 Data Model.....	16
4.5 Process Flow and Interfaces	16
4.6 Core Algorithms (Pseudo-code).....	18
4.7 User Interface Description	19
4.8 Test Plan for the Artifacts	19
Chapter 5: AI Tools Used in the Project.....	20
References.....	21

Abstract

Renting has become the primary form of housing for large segments of the Israeli population. But the market runs on contracts of adhesion. The landlord drafts the lease and hands it to the tenant on a take-it-or-leave-it basis, and a sharp information asymmetry follows. Tenants sign agreements they do not understand. They overlook the statutory protections the Israeli Fair Rental Law (5777-2017) guarantees them. They lack the time and money for an individual legal review. Tools that serve landlords are abundant. Accessible technology built to protect tenants on the consumer side (B2C) is almost entirely absent.

RightRent is an AI-based web application that reads residential rental contracts with Natural Language Processing. It is anchored in the Israeli Fair Rental Law and built on a Retrieval-Augmented Generation (RAG) architecture. The law sits in MongoDB Atlas as vector embeddings. The system matches each clause of an uploaded contract against the statutory sections closest to it. Anthropic Claude judges each clause, and a deterministic rule layer labels it as critical (red), as a warning (orange), or as compliant.

The application marks the problematic clauses on the PDF itself and points out the protections the contract leaves out. It customizes the analysis to what each tenant cares about, and it adds a negotiation agent that talks with the tenant and drafts a single reasoned WhatsApp message, consolidating all the requests, for the tenant to send to the landlord. The tenant always reviews and sends that message. The system never acts on its own.

This book sets out the needs the project answers and the goals it pursues. It examines the current situation alongside the proposed solution, and it documents the research and engineering process: the alternatives we considered, the technologies we chose, the work plan, the constraints, and the success metrics. It closes with the main work artifacts of the system: its operating logic, architecture, data model, core algorithms, user interface, and test plan.

Chapter 1: Description of Needs and Project Goals (Introduction)

1.1 Background and Current Situation

Israel's residential rental market has grown over the past few decades. A study by the Shores Institute for Socioeconomic Research, based on Israeli Central Bureau of Statistics data, shows how widespread this growth has been. The share of renters climbed from 30% to 49% among young adults aged 25-34, and from 15% to 32% among those aged 35-54 (Mishly, 2026). For a large share of the population, renting has stopped serving as a temporary stage before ownership. It has become a long-term housing arrangement. Contracts of adhesion dominate the market: one-sided agreements the landlord writes in full, leaving the tenant to accept or walk away (Kessler, 1943; Rakoff, 1983). This arrangement leaves the two sides unequally informed. The landlord, or the landlord's lawyer, drafts the agreement, knows the property, and decides what the clauses say. The tenant comes to the contract without any of that. Economists call this a textbook market failure. The tenant signs on without fully understanding the obligations, the risks, or the hidden costs (Akerlof, 1970; Garmaise & Moskowitz, 2004).

Margalit and Yizhar (2024) show that the renting population kept growing while renters themselves stayed on the margins of public debate and housing policy. The 2011 housing protests pushed rental regulation onto the agenda for a moment, but the focus soon shifted back to promoting homeownership. Renters kept the same weak bargaining position. The Knesset addressed some of these gaps in 2017 with the Rental and Lending Law, commonly called the "Fair Rental Law" (Knesset, n.d.). The law is dense, and most of the public remains unaware of what it actually grants them.

1.2 Problem Definition and Proposed Solution

The Problem: Apartment renters in Israel have no source of legal advice that is affordable, quick, and matched to their situation. Without it, they sign harsh contracts that strip away basic protections, work against their own needs, and sometimes break the Fair Rental Law outright. The typical tenant chooses among three poor options. Reviewing the contract alone demands legal knowledge most people do not have, and the tenant tends to miss something important. Hiring a lawyer costs hundreds to thousands of shekels (B144, n.d.) and takes time, which puts it out of reach for most renters. Signing with no review at all leaves the tenant exposed to void clauses and to legal and financial risk. Each path deepens the access-to-justice (A2J) problem, since much of the population cannot afford proper legal representation (Rhode, 2004).

Software for landlords, by contrast, is well developed. The Israeli platform WeCheck (WeCheck, 2024) runs AI-based background checks, identity verification, and rental guarantees. Nothing comparable exists on the tenant side. In Israel, the consumer market for tools that defend renters' rights (B2C) is still empty.

The Solution (RightRent): a web application that uses Natural Language Processing (NLP) to do the reading. Work in LegalTech has shown that current NLP systems can parse dense legal text, read it for meaning instead of matching keywords, and check clauses against a body of law with an accuracy earlier methods could not reach (Chalkidis et al., 2017; Surden, 2014). RightRent grounds that capability in the Israeli Fair Rental Law, so the analysis reflects Israeli law specifically. Contract-analysis tools such as Spellbook (Spellbook, 2024) and LawGeex (LawGeex, 2024), by contrast, serve corporate legal teams (B2B) and lawyers.

The application addresses the problem through four core mechanisms:

- **Interactive Visual Emphasizing:** the app colors suspect text on the PDF itself. Red marks a clause that breaks the law. Orange marks a clause that is legal but raises a warning or works against the preferences the tenant has set.

- **Identifying Gaps and Protections:** the app does more than summarize the lease. It finds protections the contract leaves out, such as an early-termination clause or landlord responsibility for ordinary wear and tear, and recommends adding them.
- **Personalization and Preferences:** the tenant sets priorities up front, such as a monthly ceiling, how fast repairs must be handled, or keeping a pet. The examination then centers on the risks that conflict with those priorities.
- **Efficient Communication and Negotiation:** the app includes a negotiation agent that talks with the tenant. After that conversation, it drafts a single WhatsApp message that consolidates all the requested changes. The tenant edits the message freely and sends it to the landlord.

1.3 Project Goals

At its core, the project aims to close the legal information gap between tenants and landlords in Israel. It does this through a digital tool that is cheap, quick, and within reach of ordinary renters. The goals fall into three groups.

Functional Goals: put legal knowledge within reach through an accessible LegalTech tool. Take over the contract review and spare the tenant time and money. Check that a lease complies with the Israeli Fair Rental Law. Give each tenant insights tied to their own priorities and risk profile. Support the conversation with the landlord through a negotiation agent that drafts reasoned, ready-to-send change requests.

Technological Goals: build a Retrieval-Augmented Generation (RAG) architecture on MongoDB Atlas Vector Search. Work through the difficulties of processing Hebrew, a morphologically rich language (MRL). Apply a modern stack of React, Express.js, and MongoDB.

Learning Goals: gain a deeper command of NLP, embeddings, and LLMs. Learn what it takes to fit AI into a full web system. Become familiar with where software meets the regulatory and legal world.

1.4 Structure of the Book and Literature Review

The literature review for this project is integrated into the following chapters rather than presented as a standalone section. Chapter 2 reviews the theoretical literature underpinning the problem: contracts of adhesion (Kessler, 1943; Rakoff, 1983), information asymmetry in rental markets (Akerlof, 1970; Garmaise & Moskowitz, 2004), the readability of legal language (Tiersma, 1999; Masson & Waldron, 1994), and the use of unenforceable terms in residential leases (Furth-Matzkin, 2017, 2019). Chapter 3 reviews the practical and technological literature relevant to the implementation: NLP for legal text (Chalkidis et al., 2017; Surden, 2014), retrieval-augmented generation (Lewis et al., 2020; Govea et al., 2026), the challenges of processing Hebrew as a morphologically rich language (Tsarfaty et al., 2010), and document layout analysis (Shen et al., 2021). Each source is discussed where it directly informs the analysis or an engineering decision.

Chapter 2: Problem Analysis and Solution Formation

2.1 Analysis of the Current Situation: The Israeli Rental Market and Contracts of Adhesion

Much of the problem traces back to how the Israeli rental market is structured. In several European countries, institutional and professional landlords hold a large share of the market (OECD, 2014). Israel looks different. In 2024, private individuals owned more than 81% of the apartments offered for rent (CBS, 2025), up from 78% in 2022. This points to a market that rests ever more on non-professional landlords. The lease, then, is usually a deal between two private people, a tenant and a landlord, with no professional in between to standardize it. The contracts that result are rarely uniform. Landlords often pull a template off

the internet or copy wording from one another.

In legal terms, almost the whole market runs on contracts of adhesion, the standardized agreements that the stronger party writes and the weaker party can only accept or refuse (Kessler, 1943). Writing in the Harvard Law Review, Rakoff (1983) shows how such contracts produce a market failure rooted in unequal power and unequal information.

Tenants seldom have the legal training to judge the long-term risks a lease carries, so they fall back on trusting the landlord's good faith (Garmaise & Moskowitz, 2004).

Property-management software for landlords is not in short supply. It runs background checks, weighs risk, and boosts returns. Almost nothing on the tenant side gives a renter an objective read of the risk before signing. Three kinds of products exist today, and each falls short. AI contract-analysis tools such as Spellbook and LawGeex serve corporate clients and lawyers (B2B) and know nothing of Israeli law. Israeli services such as WeCheck face the landlord, not the tenant, and offer background checks, identity verification, and rental guarantees. The remaining option is to consult a lawyer, the old route that costs a great deal and takes time.

2.2 Problem Analysis: Information Asymmetry, Legal Readability, and Conflict with the Law

Economists know this problem well. They call it asymmetric information. In his Nobel-winning paper, Akerlof (1970) showed how an imbalance of information pushes a market toward failure and leads the weaker side into poor decisions. Garmaise and Moskowitz (2004) carried the idea into real estate, where they showed that when one party lacks information, the result is mispriced deals and exploitative terms.

In an Israeli lease the imbalance shows up in three ways. The landlord usually knows the law better and knows which clauses pay off. The landlord also rents regularly and tends to keep a lawyer on call, while the tenant signs a lease perhaps once in several years. And the landlord has had time to craft a contract that serves the landlord, while the tenant meets the document for the first time and is pressed to sign on the spot.

Knowledge is not the only gap. Legal texts are hard to read, and that adds a cognitive load of its own. Language researchers have found that legalese creates a real barrier, keeping non-specialists from understanding what they have agreed to (Tiersma, 1999; Masson & Waldron, 1994). Tenants overlook harsh terms this way, and worse, they never learn about the mandatory protections the law puts in place for them.

Israel passed the Rental and Lending Law (Amendment), 5777-2017 to close these gaps. It sets minimum standards the parties cannot contract out of. The security deposit may not exceed one-third of the rent for the lease term or three months' rent, whichever is lower. The landlord must repair urgent faults within a set period. And a habitable apartment must meet a basic floor that covers drainage, water, electricity, ventilation openings, and an entrance door (Knesset, n.d.).

Scholars who have read the law closely note that questions of interpretation and enforcement linger even with these mandatory protections in place. The statute on paper still differs from the statute in practice in the private market (Levin-Schnur & Procaccia, 2018). Landlords, meanwhile, keep writing clauses that run against the law (Furth-Matzkin, 2017). Such clauses are void, yet their presence alone has a chilling effect. A tenant who does not know the law stays quiet and never asks for the protections owed to them (Furth-Matzkin, 2019). On top of this, the cost of a lawyer looks too high against the risk a tenant senses in the moment. The tenant enters the negotiation with no protection at all.

2.3 Solution Formation: Semantic Analysis, Personalization, and Technological Considerations

RightRent rests on current AI and Natural Language Processing models. Unlike a simple keyword matcher, which only looks for words, NLP systems in LegalTech read a clause for

its meaning, extract what it requires from each side, and check those against a database of statutes with an accuracy keyword methods never had (Chalkidis et al., 2017; Surden, 2014). What sets RightRent apart from tools like Spellbook and LawGeex is whom it serves. Those tools work for corporate legal teams (B2B) reviewing many contracts, run on playbooks that lawyers write in advance, and stay neutral to any one country's law. RightRent works for a single Israeli tenant (B2C) and is anchored in the Israeli Fair Rental Law.

Why RAG: the system uses a Retrieval-Augmented Generation (RAG) architecture, which pairs vector embeddings with Large Language Models (LLMs). We weighed it against two alternatives: prompt engineering only, sending the whole contract and the whole law to the model on every call, and fine-tuning a model on tagged contracts. We chose RAG for four reasons. We do not have labeled data on the scale fine-tuning would need. The Fair Rental Law is fixed and well defined, and that suits storing it ahead of time in a vector database. When the law changes, the knowledge base can be updated without retraining. And RAG holds down hallucinations in legal work, where a wrong answer carries a cost. Govea et al. (2026), published in *Frontiers in Artificial Intelligence*, back this up. They found that in the legal domain, RAG architectures outperform generic LLMs, especially when an answer must be grounded in the exact text of the law.

Hebrew NLP Challenges: researchers widely classify Hebrew as a morphologically rich language (MRL). Its dense morphology, its prefixes and suffixes that shift meaning, and its word boundaries that do not line up with meaning all demand careful handling. The project relies on a multilingual embedding model that supports Hebrew, and Claude handles the deeper legal interpretation.

General Architecture: the system has a React frontend, an Express.js server, and a MongoDB database. MongoDB earns its place here because MongoDB Atlas carries Vector Search of its own, so the Fair Rental Law embeddings sit in the same store as the user data and the contracts. No separate vector database such as Pinecone or Chroma is needed. That keeps the architecture and the deployment simpler. The solution works on four fronts. The first is risk identification: the system reads the PDF and marks clauses that break the Fair Rental Law. The second is identifying gaps and missing protections: the system notes protections the contract leaves out so it can suggest adding them. The third is a personal risk profile. Claude receives the tenant's stated priorities alongside each clause and judges whether the term conflicts with them, and a deterministic rule layer turns that verdict into the final label. Orange marks a warning, a term that is legal but looks risky or cuts against what the tenant wants. Red is reserved for clauses that break the law. The fourth is help with the negotiation itself. The power gap and the fear of confrontation keep tenants silent, so the app includes a negotiation agent that drafts a single consolidated WhatsApp message. Working from the negotiation principles of Fisher, Ury, and Patton (2011), it drafts a calm, factual message built on the tenant's legal rights. That approach makes the landlord more likely to agree and leaves the tenant less anxious about asking.

Chapter 3: Research and Engineering Process

3.1 Review of Research and Engineering Challenges

Building RightRent, a system that reads Hebrew rental contracts, raises problems in several areas at once: Hebrew NLP, a RAG-based AI design, structured PDF processing, privacy, and a B2C interface that ordinary users can use. This section lays out the main ones, the options we weighed, and the calls we made. **Legal accuracy:** this is the central challenge. Reading a contract for legal trouble leaves little room for mistakes, since a false positive or a false negative can send the tenant the wrong way. In ordinary NLP, 85% accuracy counts as good. Here, each miss can do harm, so the engineering has to be more careful. **A morphologically rich language:** the literature classes Hebrew as a morphologically rich language. In Hebrew, words such as "and," "in," and "to" attach to the next word instead of

standing on their own, so tokenizers built for English break down. Tsarfaty et al. (2010) set out why. Legal Hebrew makes it harder still, with archaic phrasing and long sentences, and the absence of vowel marks (niqqud) leaves some words open to more than one reading. The pipeline handles this with a multilingual embedding model that covers Hebrew for retrieval, and Claude, which handles Hebrew well, takes on the understanding itself.

Document structure: PDFs, especially scanned PDFs, do not store the contract in a clean structure. They often lose information about paragraphs, headings, sections, and sub-sections. Simple OCR would give us only one messy block of text, with no clear clause boundaries. To solve this, we use layout analysis ideas, similar to Layout Parser, inside our Node pipeline. The goal is not only to extract the text but also to keep the structure of the contract: where each clause starts and ends, how the document is visually organized, and the exact position of each word. This matters because the system later needs to highlight the relevant parts of the contract for the user. **Privacy:** a lease holds sensitive details, names, ID numbers, and addresses, and sending them to a cloud LLM is a risk the system has to handle. **Keeping the law current:** the Fair Rental Law can change, so the system has to let its knowledge base be updated without retraining anything.

3.2 Detailed Requirements

3.2.1 Functional Requirements

1. The tenant shall be able to register an account.
2. The tenant shall be able to log in to the system.
3. The tenant shall be able to manage their preferences.
4. The tenant shall be able to edit their preferences.
5. The tenant shall be able to delete their analysis history.
6. The tenant shall be able to upload a rental contract.
7. The tenant shall be able to request an analysis of the uploaded contract.
8. The tenant shall be able to confirm their preferences before the analysis runs.
9. The tenant shall be able to view the results of the contract analysis.
10. The tenant shall be able to view the marked-up PDF of the contract.
11. The tenant shall be able to prepare a negotiation message based on the analysis results.
12. The tenant shall be able to review the WhatsApp message before sending it.
13. The tenant shall be able to edit the WhatsApp message.
14. The tenant shall be able to view their analysis history.
15. The system shall be able to update the law knowledge base automatically through an external API.

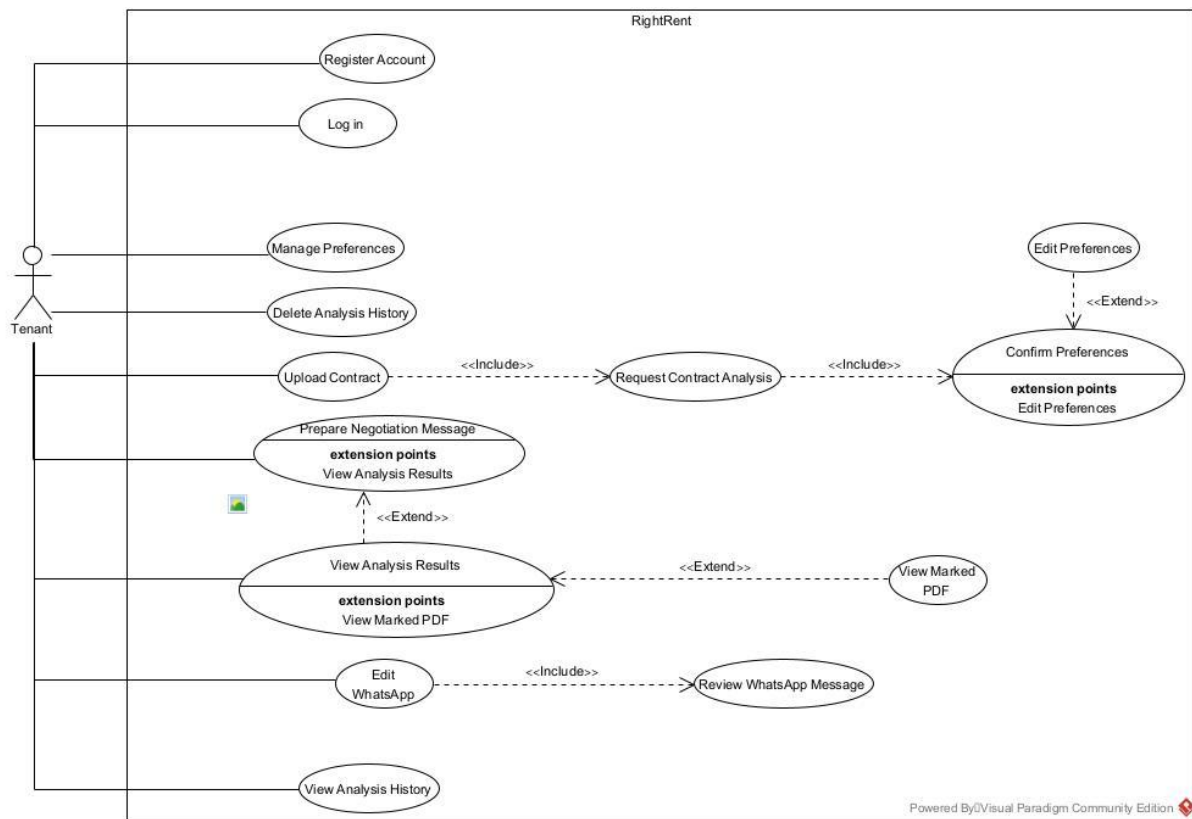


Figure 3.1 Use Case Diagram of the RightRent system (actors, use cases, and «include» / «extend» relationships)

3.2.2 Non-Functional Requirements

- **Performance:** a 1-6 page contract analyzed in under 90 seconds, with 10 concurrent users supported.
- **Security:** bcrypt password hashing, HTTPS only
- **Privacy:** personal information anonymized before it reaches the LLM, and a way to delete history.
- **Compatibility:** modern browsers and mobile devices, through responsive design.
- **Legal Currency:** an architecture that lets the knowledge base be updated without retraining.

3.3 Research Approaches Considered

We considered three ways of bringing AI into the system before settling on one.

Approach 1 (prompt engineering only): send the whole contract and the whole law to the LLM on every call. This is easy to build, but the costs run high. Each call uses many tokens. The model may miss details in long inputs because of the "lost in the middle" problem (Liu et al., 2024). It may give incorrect answers, and updating the law is difficult. This approach is not strong enough for a system that has to be legally accurate.

Approach 2 (fine-tuning): train an existing pretrained language model on tagged contracts. Done well, this could be fast and accurate. But it needs a large body of tagged data we do not have, costs a lot of compute, makes the law hard to update, and offers little transparency. A graduation project has neither the resources nor the time for it.

Approach 3 (retrieval-augmented generation, the one we chose): store the law ahead of time as vector embeddings, and for each contract clause, retrieve only the relevant legal clauses. This keeps the analysis tied to the real text. Legal accuracy rises, incorrect AI

answers drop, costs stay low, updates come easily, and the system stays transparent. The price is a more involved architecture, and we took it anyway. In the original RAG paper, Lewis et al. (2020) showed that joining the model's own memory with retrieval from a database lowers hallucinations and improves accuracy on knowledge-heavy tasks. Govea et al. (2026) confirm the same in the legal domain.

3.4 Technology Selection

Main LLM (Anthropic Claude): we compared OpenAI GPT, Google Gemini, DeepSeek, and several open models, and picked Claude. It handles Hebrew well, performs well on legal tasks with fewer hallucinations, follows involved instructions, and runs on US servers. That last point matters for privacy in a way Chinese models such as DeepSeek do not settle.

Embeddings (OpenAI text-embedding-3-small): it is cheap, about \$0.02 per million tokens (OpenAI, n.d.), covers many languages including Hebrew, and integrates easily into the system.

PDF processing (pdfjs-dist): this is the Mozilla library behind PDF.js. It is free, runs in Node.js where PyMuPDF (a Python tool) would not, extracts text with exact coordinates, and copes with Hebrew and right-to-left text. Scanned contracts go through Tesseract.js for OCR, and the layout-analysis ideas from Shen et al. (2021) guide how we keep the logical structure.

Marked-PDF generation (pdf-lib): a Node.js library that adds annotations, the color marks, onto an existing PDF without disturbing its original structure.

Anonymization (regular expressions plus Hebrew NER): a layer with two passes runs inside the Node pipeline. The first pass uses deterministic regular expressions to catch structured identifiers such as ID numbers, phone numbers, and addresses. Sums of money stay intact on purpose, because the analysis depends on them: the deposit-cap check and the comparison against the tenant's budget preference both need the actual amounts. The second pass uses a pretrained Hebrew NER model, such as DictaBERT-NER, to catch personal names that the patterns miss. Both passes run locally on the server, so no personal detail leaves the system before redaction.

General stack: a React frontend, an Express.js and Node.js backend, a MongoDB Atlas database with built-in Vector Search, and authentication through JWT.

3.5 Architecture and Analysis Process

3.5.1 System Components

The system has eight components across three tiers. A React client renders the interface. An Express API gateway handles requests and authentication. User management, with its data in MongoDB Atlas, keeps accounts, preferences, and history, and the same store also holds the contracts and the Fair Rental Law embeddings. A PDF extraction service (pdfjs-dist) extracts text and per-word coordinates, with Tesseract.js OCR for scans. An anonymization service strips personal information before anything reaches the LLM. A RAG and risk-scoring component creates embeddings with OpenAI text-embedding-3-small, retrieves from MongoDB Vector Search, and runs the two-track analysis with Claude, with a deterministic rule layer assigning the final label. A results/report generation service produces the marked PDF and the dashboard. And a negotiation agent, built as a state machine, guides the tenant through the negotiation and drafts the message.

3.5.2 Stages of the Analysis Process

- **Stage 1, upload and validation:** the user uploads a PDF, and the system checks its format and size (up to 10 MB).
- **Stage 2, text and structure extraction:** pdfjs-dist pulls out the text with the (x, y, page) coordinates of each word. A scanned file triggers OCR. Layout analysis keeps the contract's logical structure.

- **Stage 3, anonymization:** before anything goes to the LLM, a set of regular expressions and a Hebrew Named Entity Recognition (NER) model find personal details, names, ID numbers, and addresses, and swap them for generic placeholders such as [ID_NUMBER] or [LANDLORD_NAME].
- **Stage 4, chunking:** the text is split along the contract's own structure into clause-sized chunks averaging 200-300 words.
- **Stage 5, retrieval:** each chunk is embedded, and a cosine-similarity search in MongoDB Vector Search matches it against the Fair Rental Law embeddings, returning the five closest statutory clauses (top-k = 5).
- **Stage 6, generation:** Claude gets a structured prompt with the chunk, the relevant law clauses, and the tenant's preferences. Chain-of-Thought makes it justify each flaw from the law, and a Self-Verification step has the model check its claims against the source text before it commits to an answer. The judgment runs on two tracks: whether the clause is legal, and whether it is legal but still risky or in conflict with the tenant's preferences. Claude returns only a structured verdict and does not assign the final label. A deterministic rule-based function, `classifyClause` (Algorithm 2), turns the verdict into red when the clause breaks the law, or orange when the clause is legal but raises a warning or conflicts with the tenant's preferences.
- **Stage 7, contract level protection checklist:** once the system classifies every chunk, the backend sends the anonymized contract clauses and a predefined protection checklist to Claude. The checklist contains legal protections and recommended additions for residential leases, such as early termination, repair responsibility, ordinary wear and tear, and tenant preference items. Claude returns JSON for each checklist item with one of three statuses: covered, partial, or missing, plus relevant clause IDs when the contract contains evidence. The backend adds missing and partial items to the suggested additions list. It marks an item RED only when an actual contract clause violates the Fair Rental Law.
- **Stage 8, results display:** the system maps the findings back to their coordinates in the original PDF, generates the marked copy with pdf-lib, stores the findings, protection report, suggested additions, and marked PDF reference in MongoDB Atlas Database, and then loads the saved analysis results for the dashboard.

3.5.3 Knowledge-Base Preparation The legal knowledge base is prepared through an API-based synchronization process. During the initial system setup, and later whenever the legal source is checked for updates, the RightRent backend fetches the Fair Rental Law from a configured legal source API. The Law Knowledge Sync Service parses the returned HTML or text into structured legal sections and subsections, normalizes the text, validates that the required sections were extracted correctly, and then creates embeddings for each law chunk using OpenAI text-embedding-3-small. Each law chunk is stored in MongoDB Atlas Vector Search with its embedding vector and metadata such as section number, title, source URL, content hash, and update date. The MongoDB Atlas Database stores synchronization metadata, including the source URL, sync status, section count, update timestamp, and active version. This process is separate from contract analysis. Contract analysis uses the already prepared vector index, while the sync process prepares and updates that index. This allows the legal knowledge base to change without retraining the model.

3.6 The Negotiation Agent

RightRent does more than analyze a contract once. It also has a negotiation agent, an interactive part that walks the tenant through building a strategy for talking to the landlord. All message drafting runs through the agent, and the guided conversation ends in a single consolidated WhatsApp message that covers every request the tenant chose to raise.

The need for the agent: Earlier chapters described the gap between tenant and landlord as legal, but it is psychological too. Plenty of tenants would rather not push back at all. Drawing on the negotiation principles of Fisher, Ury, and Patton (2011), the agent helps a tenant

come to the conversation with confidence and a clear case.

Operation: The agent talks only with the tenant and never contacts the landlord. The conversation runs in stages. The agent first helps the tenant rank the issues by what matters most. For each issue it offers a few ways to play it, such as citing the law directly or proposing an alternative. Once the tenant has chosen a strategy for every issue, the agent drafts one consolidated WhatsApp message that follows all of those choices, and it also prepares replies to the answers the landlord is likely to give. The tenant reviews the message, edits it freely, and sends it to the landlord themselves.

Technical implementation: The agent is a state machine that keeps the conversation state in MongoDB, with its own prompt to Claude at each stage. We built it this way rather than reaching for a heavier framework such as LangChain, so the logic stays transparent and under our control. It extends the RAG servers already in place and reuses the same components.

3.7 Work Plan and Timeline

Development runs across two semesters with a break in between, about six active months, for a team of two students.

- **Stage 1, research and design (weeks 1-3):** study the Fair Rental Law, design the architecture, choose the libraries, and prepare the law embeddings.
- **Stage 2, basic servers (weeks 4-7):** stand up the backend and MongoDB, build the user system, and put together the React skeleton and the upload screen.
- **Stage 3, contract-analysis core (weeks 8-14):** PDF processing, anonymization, the RAG pipeline, the Claude integration, the prompts, and testing on real contracts.
- **Break between semesters:** gather feedback, write documentation, and study.
- **Stage 4, advanced features (weeks 15-21):** the preferences system, the dashboard, the marked PDF, the negotiation agent, and the analysis history.
- **Stage 5, testing and defence (weeks 22-26):** thorough testing, optimization, UX work, and getting ready for the presentation and defence.

Role division: Student 1 takes the frontend and UX, including the server-side marked-PDF generation module (pdf-lib), since it ties tightly to the results display. Student 2 takes the backend, RAG, and AI integration. Both students build the AI components and the agent together.

3.8 Constraints, Risks, and Mitigation Strategies

- **Legal accuracy and hallucinations:** A wrong reading could hurt the tenant. The RAG architecture, the Self-Verification step, and a bias toward high recall guard against this. Flagging a harmless clause as a warning is safer than missing a dangerous one.
- **Unauthorized practice of law (UPL):** The app cannot stand in for a licensed lawyer. It keeps a human in the loop: it only suggests, never edits a contract or sends a message on its own, and it states plainly that its output is a helper, not binding legal advice.
- **Data privacy and security:** Contracts hold sensitive personal data. An anonymization pipeline (regular expressions plus Hebrew NER) runs before anything reaches the LLM and swaps identifying details for generic placeholders.
- **Hebrew processing quality:** Tools for Hebrew legal text trail those for English. We test thoroughly on Hebrew legal text and rely on models with strong Hebrew support, OpenAI's multilingual embeddings and Claude. If retrieval quality proves weak, a Hebrew-specific encoder remains a possible fallback.
- **API costs and the lack of tagged data:** Every LLM call costs money, and no tagged contract repository exists. We cache results, tune the chunk size, and develop on Claude Haiku. We also build a test set of 10 to 20 contracts by hand, with a lawyer's input where we can get it.

- **Risk:** The project is ambitious, so the priorities are firm. The core analysis pipeline comes first. The advanced features, the interactive negotiation agent among them, come after.

3.9 Success Metrics

We judge the project on four fronts: legal accuracy, performance, privacy and security, and the quality of the negotiation agent. Each target below maps to a test case in Section 4.8.

Legal accuracy: We measure the classifier against a hand-built test set of 10 to 20 real rental contracts, annotated in advance and reviewed with legal input where we can get it. On this set the system must reach recall of at least 90% on Fair Rental Law violations, with precision of at least 95% on red and orange labels. The gap between the two thresholds follows from the cost of each error. A false alarm costs the tenant a moment of attention. A missed violation costs the tenant money or rights, so the pipeline leans toward recall. One failure mode we rule out: no illegal clause may carry an OK label (T3, T5). The missing-protection detector must find at least 85% of the protections we removed on purpose from the test contracts (T10).

Performance: A contract of 1 to 6 pages, marked PDF included, returns in under 90 seconds, and the system holds that target with 10 users analyzing contracts at once (T7, T11). Uptime during the deployment period stays at 99% or above.

Privacy and security: No name, ID number, or address reaches the LLM. We verify this on the test set and accept nothing short of full redaction (T2). A prompt injection embedded in a contract must not alter the verdict, and the API must block XSS and CSRF attempts (T9, T12).

Negotiation agent: Every conversation must run the state machine from PRIORITIZE to DONE without an invalid transition (T6), and every legal claim in the drafted WhatsApp message must trace back to a specific section of the law.

Beyond the system itself, we set two learning goals: finishing on schedule, and showing command of RAG, LLMs, and agents at the defence.

Chapter 4: Work Artifacts

4.1 Overview of Project Deliverables

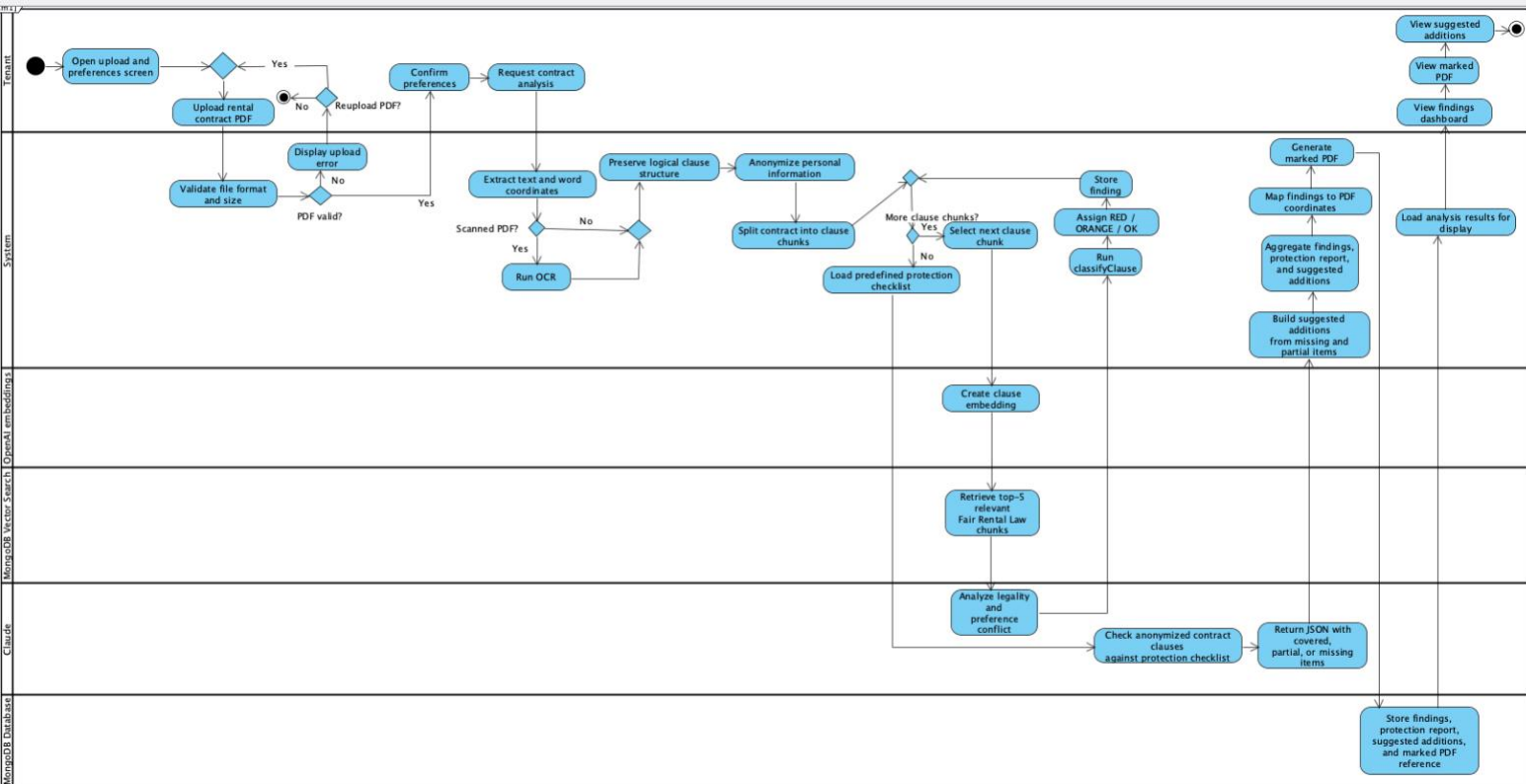
The deliverable is RightRent, a full-stack web system in three tiers: a React client, an Express.js and Node.js server, and a MongoDB Atlas database with built-in Vector Search. It builds on the eight components from Chapter 3, and the main artifacts are these:

- **React client:** the three-screen interface (upload/preferences, analysis/dashboard, negotiation).
- **Express API gateway:** REST endpoints with JWT authentication.
- **User management & MongoDB:** sign-up/login (JWT + bcrypt), preferences, and analysis history in MongoDB Atlas.
- **PDF extraction service:** text + coordinate extraction with pdfjs-dist, Tesseract.js OCR for scans.
- **Anonymization service (regular expressions plus NER):** redacts identifying data before the LLM.
- **RAG and risk scoring:** embeddings from OpenAI text-embedding-3-small, retrieval from MongoDB Vector Search, and Claude analysis with a deterministic rule layer (classifyClause) that labels each clause RED/ORANGE/OK by legality and personal fit.
- **Results/report generation:** a color-marked PDF (pdf-lib) and a findings dashboard.
- **Negotiation agent (state machine):** guides the tenant through building a strategy, and at the end of the conversation it drafts a single consolidated WhatsApp message for the tenant to send.

4.2 Activity Diagram

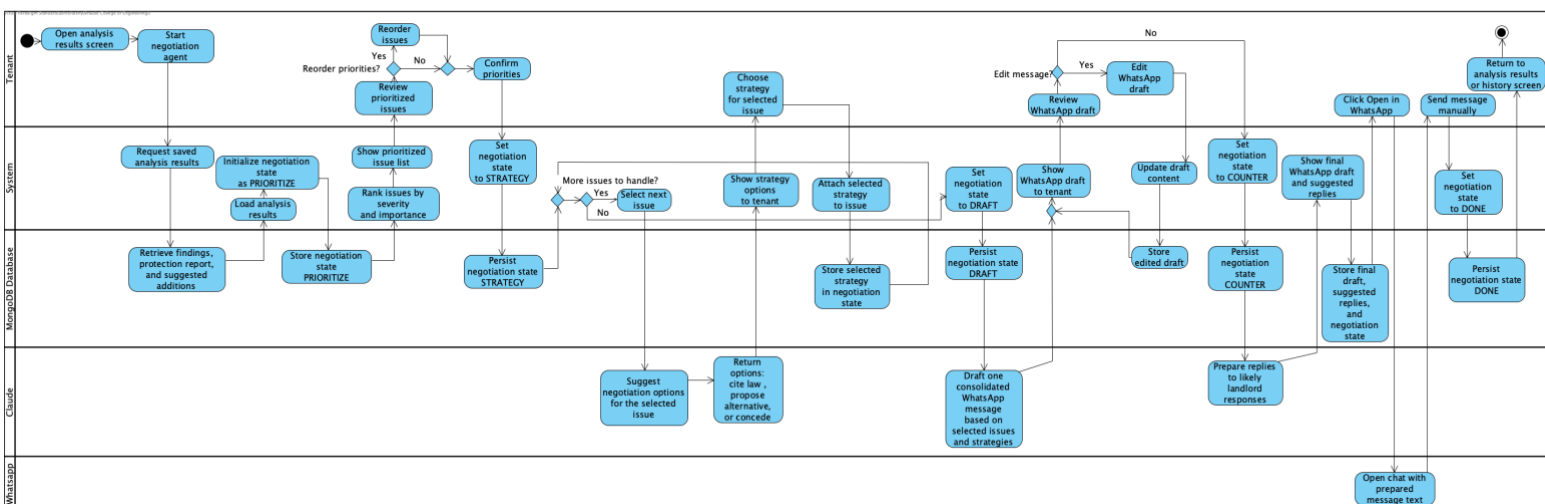
4.2.1 Scenario 1: Contract Upload and AI Lease Analysis

Goal: Enable the tenant to upload a residential rental contract PDF, set personal preferences, and receive an AI-based legal analysis that identifies illegal clauses, risky clauses, missing protections, suggested additions, and a marked PDF based on the Israeli Fair Rental Law.



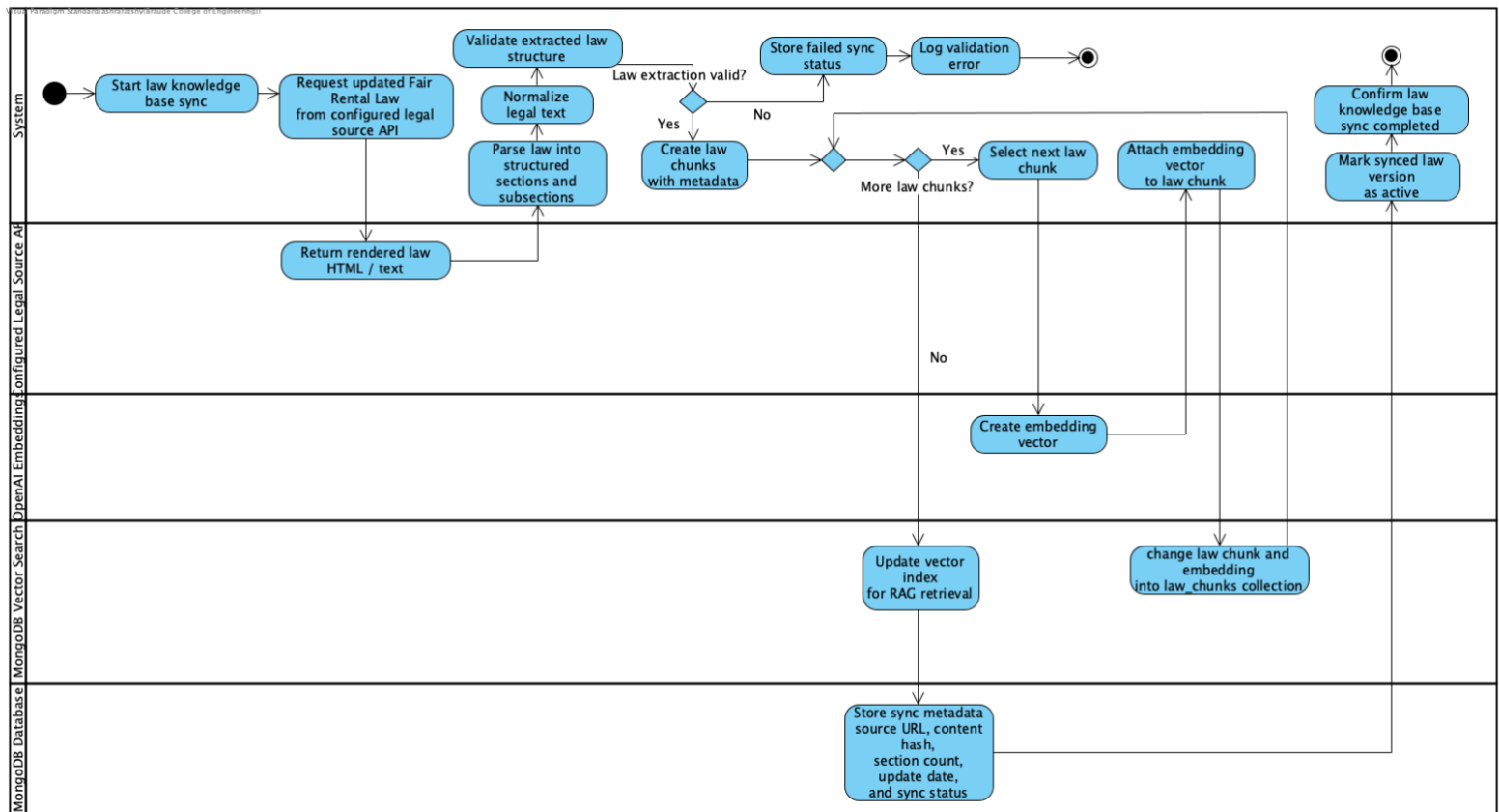
4.2.2 Scenario 2: Negotiation Agent and WhatsApp Message Preparation

Goal: Enable the tenant to review the analysis results, prioritize the detected issues, choose a negotiation strategy for each issue, and generate an editable WhatsApp message that the tenant can manually send to the landlord.



4.2.3 Scenario 3: API-Based Law Knowledge Base Synchronization

Goal: Enable the RightRent System to update the Fair Rental Law knowledge base by fetching the updated law text from a configured legal source API, parsing it into structured law sections, creating embeddings, and storing the law chunks in MongoDB Atlas Vector Search for future RAG retrieval.



4.3 System Architecture and Software Structure

RightRent runs on three tiers that keep presentation, logic, and data apart. The client tier is a React frontend for the tenant flow: uploading contracts, setting preferences, viewing results, using the negotiation agent, and checking history. The law knowledge base is updated by a backend synchronization service, separate from the tenant interface.

The server tier is an Express.js/Node.js backend behind a REST API with JWT authentication. It is organized into three main services. The Contract Analysis Service handles PDF extraction, anonymization, RAG analysis, risk classification, and marked-PDF generation. The Negotiation Agent Service guides the tenant through the negotiation flow and drafts the WhatsApp message. The Law Knowledge Sync Service fetches the Fair Rental Law from a configured legal source API, parses it into structured law clauses, validates the result, creates embeddings, and stores them in MongoDB Atlas Vector Search. The data tier is MongoDB Atlas, split into Atlas Database and Atlas Vector Search. Atlas Database stores users, contracts, analyses, negotiations, and law-sync metadata. Atlas Vector Search stores the law_chunks collection with embedding vectors and a vector index used by the RAG retrieval step.

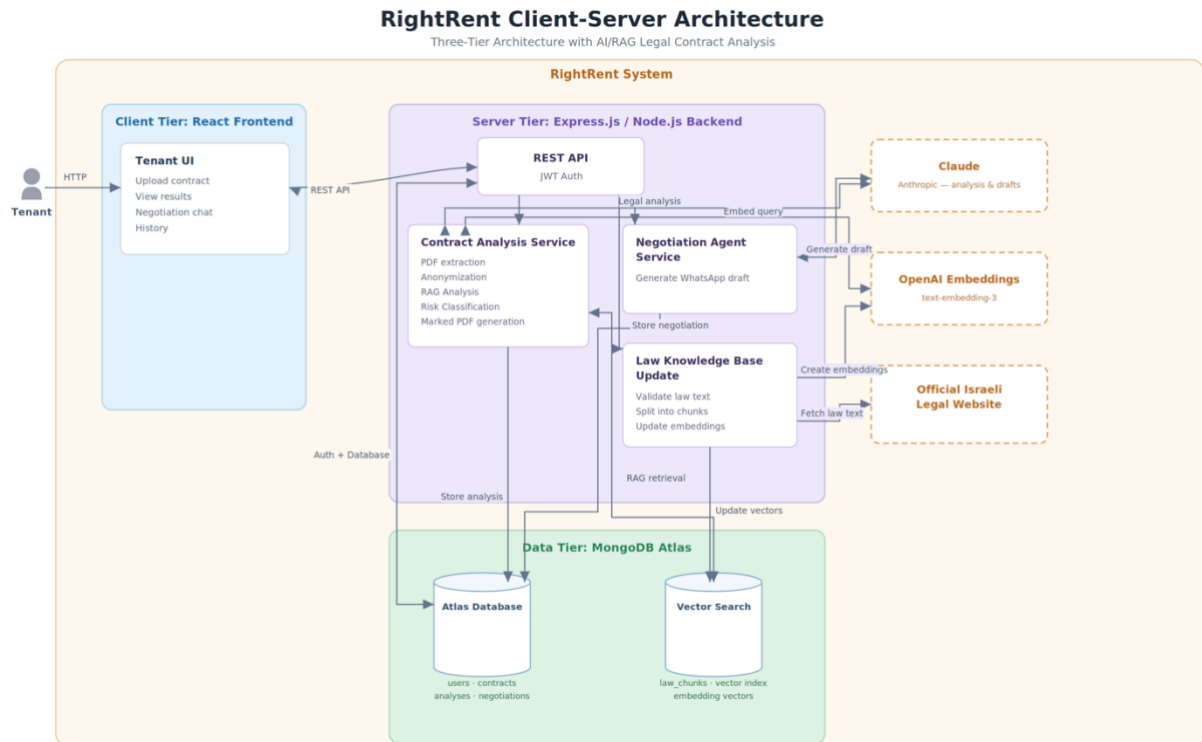


Figure 4.2 System structure (three-tier architecture, eight components)

4.4 Data Model

Five MongoDB collections hold the data. The users collection keeps account details and preferences. The contracts collection holds each contract broken into clauses with their coordinates. The analyses collection stores the findings and the missing protections. The negotiations collection records the agent's conversation state as a state machine. And law_chunks holds the statute sections with their embedding vectors under a vector index. Figure 4.3 gives the UML class diagram.

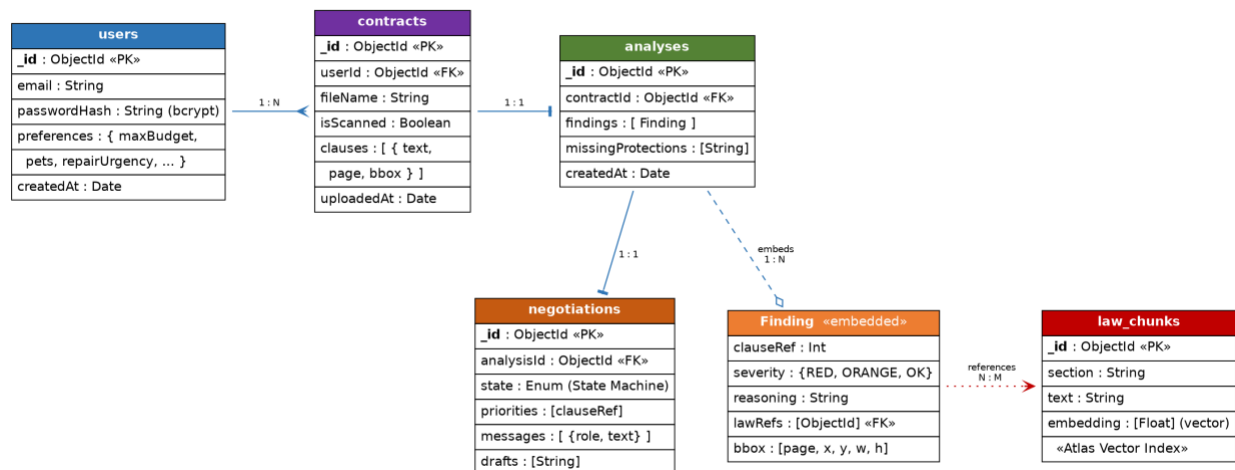


Figure 4.3 Data model (UML) and MongoDB collection structure

4.5 Process Flow and Interfaces

The client and server talk over a REST API with JWT authentication. The sequence diagram in Figure 4.4 follows how the components interact during an analysis,

including the split between the embeddings service (OpenAI) and the analysis model (Claude).

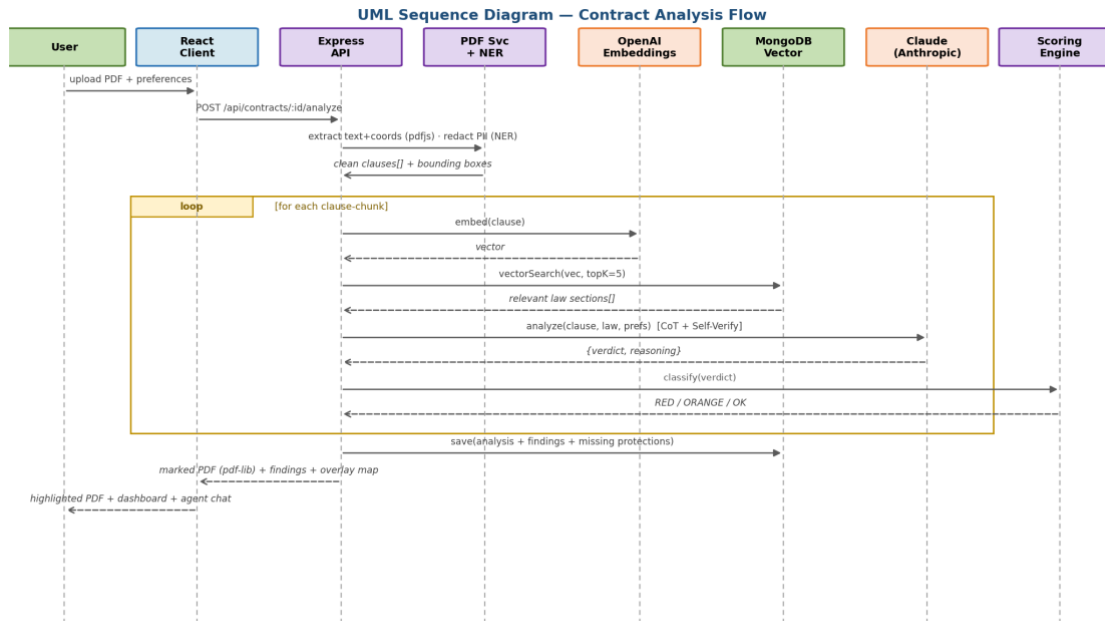


Figure 4.4 UML sequence diagram of the analysis flow

The main API endpoints appear in the table below.

Endpoint	Method	Description
/api/auth/register · /login	POST	Sign-up / login (JWT + bcrypt)
/api/users/preferences	PUT	Update profile and preferences
/api/contracts/upload	POST	Upload PDF, validate, extract structure
/api/contracts/:id/analyze	POST	Run the RAG + scoring pipeline
/api/analyses/:id	GET	Retrieve findings, marked PDF, dashboard
/api/negotiation/:analysisId	POST	Advance the negotiation-agent conversation
/api/history	GET	The user's analysis history
/api/history/:id	DELETE	Delete an analysis from history
/internal/law/sync	POST	Trigger law knowledge base synchronization (internal maintenance only)

4.6 Core Algorithms (Pseudo-code)

Four algorithms sit at the heart of the system: the analysis orchestration that drives the main flow, the scoring algorithm that classifies a clause on two dimensions, the highlight mapping that ties Claude's output back to PDF coordinates, and the negotiation agent that runs the interactive state machine.

Algorithm 1 analyzeContract(pdf, preferences)

```

clauses <- PDFExtractor.extract(pdf, lib='pdfjs-dist') // text + coords; OCR if scanned
clean <- NER.redactPII(clauses) // anonymize before cloud LLM
findings <- []
FOR EACH chunk IN chunkByClause(clean, ~200-300 words) DO
  vec <- OpenAI.embed(chunk.text) // text-embedding-3-small
  lawSections <- MongoDB.vectorSearch(vec, topK = 5) // cosine similarity
  verdict <- Claude.analyze(chunk, lawSections, preferences) // CoT + Self-Verify
  severity <- classifyClause(verdict)
  findings.append({ chunk, severity, reasoning: verdict.reason })
END FOR
missing <- detectMissingProtections(clean, lawDB)
overlay <- mapHighlightsToPDF(findings, clauses) // pdf-lib annotations
RETURN { findings, missing, overlay } // agent + human act on this

```

Algorithm 2 classifyClause(verdict)

```

// Claude judges both tracks; this deterministic rule layer only assigns the label
// Vector A: objective legality (vs Fair Rental Law, retrieved by RAG)
IF verdict.violatesLaw == TRUE THEN
  RETURN RED // illegal / void clause -> critical
END IF
// Vector B: legal-but-risky warning, or conflict with user preferences
// (both tracks are judged by Claude, which receives the tenant's preferences,
// maxBudget / pets / repairUrgency, in its structured prompt)
IF verdict.riskWarning == TRUE THEN RETURN ORANGE // legal but risky
IF verdict.preferenceConflict == TRUE THEN RETURN ORANGE // personal conflict
RETURN OK // compliant + aligned
// Tuning: biased toward High Recall - borderline cases escalate to ORANGE.

```

Algorithm 3 mapHighlightsToPDF(findings, clauses)

```

overlay <- []
FOR EACH f IN findings DO
  IF f.severity != OK THEN
    box <- clauses[f.clauseRef].bbox // [page, x, y, w, h] from pdfjs-dist
    color <- (f.severity == RED) ? '#C00000' : '#ED7D31'
    pdfLib.drawRectangle(box.page, box, color, opacity=0.35)
    pdfLib.attachNote(box, f.reasoning)
  END IF
END FOR
RETURN pdfLib.save() // marked PDF preserving the original layout

```

Algorithm 4 negotiationAgent(analysis) [State Machine]

```

state <- PRIORITIZE           // persisted in MongoDB 'negotiations'
WHILE state != DONE DO
  SWITCH state:
    PRIORITIZE: agent ranks findings by severity; user reorders -> STRATEGY
    STRATEGY : FOR EACH issue, Claude proposes options
                 {cite-law | propose-alternative | concede}; user picks;
                 more issues? stay in STRATEGY else -> DRAFT
    DRAFT  : Claude drafts ONE consolidated WhatsApp message covering all
              chosen requests (Fisher-Ury principles); user edits freely -> COUNTER
    COUNTER : Claude pre-drafts replies to likely landlord responses
              -> DONE
  persist(state, conversation)
END WHILE
RETURN draft // a single message; nothing is sent automatically (Human-in-the-loop)

```

4.7 User Interface Description

The interface keeps friction low across three main screens. On the first, upload and preferences, the tenant drags in a lease PDF of up to 10 MB and sets preferences: a maximum monthly budget, repair urgency (24 hours, 48 hours, or one week), lease length, a rent-increase cap, and toggles for pets, furnishing, and a guarantor requirement. The second screen, analysis and dashboard, shows the marked contract with color-coded bars next to a findings summary that counts critical issues, warnings, and compliant clauses. It also includes a detail card for the selected finding and a list of suggested additions such as an early-termination clause or a wear-and-tear clause. The third screen, the negotiation agent, runs a staged chat with the tenant. The agent surfaces the issues one at a time, the tenant responds with how to handle each, and the agent consolidates everything into a single WhatsApp draft. The tenant reviews that draft and sends it through the "Open in WhatsApp" button, keeping a human in the loop.

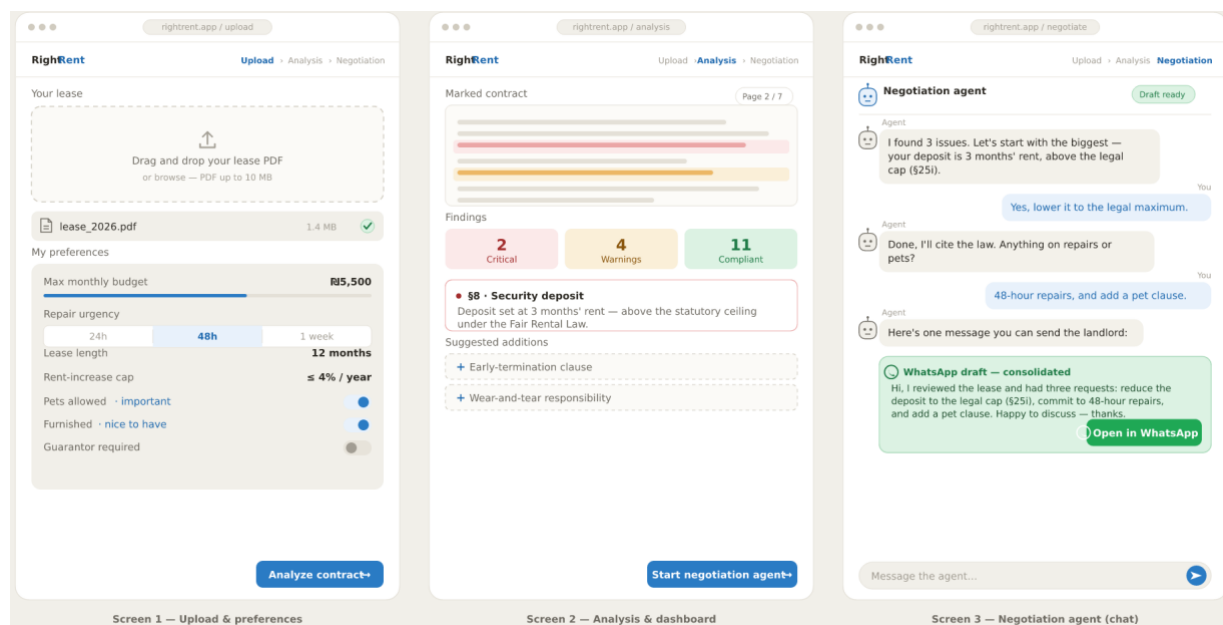


Figure 4.5 UI wireframe (the three core screens)

4.8 Test Plan for the Artifacts

The plan mixes unit, integration, and end-to-end tests. Because this is a legal tool, it weighs accuracy and recall most heavily, since flagging a harmless clause as a warning beats missing a dangerous one, and it checks the Chapter 3 performance

target of an analysis inside 90 seconds, along with the load and security requirements (10 concurrent users, XSS and CSRF). The 99% availability target is verified through uptime monitoring during the deployment period rather than through a single test case. The table below lists the main test cases.

#	Type	Test Scenario	Expected Result
T1	Unit	Clause extraction from PDF (pdfjs-dist)	All clauses extracted with valid coordinates
T2	Unit	Anonymizing ID numbers and names (NER)	All PII replaced with generic placeholders
T3	Unit	Deposit exceeds the lawful cap (lower of one-third of total rent or 3 months' rent)	Classified RED (legal violation)
T4	Integration	Pet ban vs. pet preference	Classified ORANGE (personal conflict)
T5	Integration	RAG retrieval (top-k = 5) returns statute	High recall on the test set
T6	Integration	Negotiation-agent state transitions	Valid PRIORITIZE → ... → DONE flow
T7	Performance	Analyze a rental contract of 1 to 6 pages	Completes within 90 seconds
T8	E2E	Upload → analyze → marked PDF → message	Full flow completes without errors
T9	Security	Prompt injection inside the contract	Malicious in-text instructions ignored
T10	Integration	Detect missing protections in a contract that leaves out required terms	All missing protections found and listed
T11	Performance	Load: 10 concurrent users each analyzing a contract	All analyses complete correctly within the performance target
T12	Security	XSS and CSRF attempts against the web interface and API	Malicious requests blocked; no script runs and no state changes without a valid token

Chapter 5: AI Tools Used in the Project

We used Anthropic Claude as an assistant while researching and building the project. It helped in a few ways. It clarified ideas in Natural Language Processing, Retrieval-Augmented Generation, and LegalTech. It reviewed the system architecture and the pseudo-code with us. And it helped draft and edit the English of this book.

5.1 AI Prompts

- Sort the article references by A-Z:
Ref: <https://claude.ai/share/d49587cf-d63d-4746-baa8-270c3f58a752>
- write me 1 paragraph about this article:
Chalkidis, I., Androutsopoulos, I., & Michos, A. (2017). Extracting contract elements. *Proceedings of the 16th International Conference on Artificial Intelligence and Law (ICAIL '17)*, 19-28.
Ref: <https://claude.ai/share/d3cba934-1b1b-41a7-aa68-6783ff86ffd3>
- Summarize: Kessler, F. (1943). Contracts of adhesion, Some thoughts about freedom of contract. **Columbia Law Review*, 43*(5), 629-642.
Ref: <https://claude.ai/share/a8c399bf-a8de-4065-8f57-267ed1e4c078>
- Summarize: OECD. (2014). **A revival of the private rental sector of the housing market?** OECD Economics Department Working Papers No. 1170. Paris: OECD Publishing.
Ref: <https://claude.ai/share/88ddd273-a832-40e6-9e5c-f73f477c4b9c>

References

References are listed alphabetically by author surname, following APA style. Hebrew-language sources are marked accordingly.

- Akerlof, G. A. (1970). The market for "lemons": Quality uncertainty and the market mechanism. *The Quarterly Journal of Economics, 84*(3), 488-500.
<https://www.jstor.org/stable/1879431>
- B144. (n.d.). Real-estate lawyer: Rental transactions. B144 Magazine. Retrieved June 30, 2026, from <https://www.b144.co.il/Magazine/שכירות-עסקאות-שכירות/עורך-דין-מקרקעין/עסקאות-שכירות-שכירות/>
- Central Bureau of Statistics. (2025). *Dwellings and buildings in Israel, 2024*. Jerusalem: CBS. [Hebrew]. <https://www.cbs.gov.il/he/mediarelease/pages/2025/-דירות-ומבנים-2024.aspx>
- Chalkidis, I., Androutsopoulos, I., & Michos, A. (2017). Extracting contract elements. *Proceedings of the 16th International Conference on Artificial Intelligence and Law (ICAIL '17)*, 19-28. <https://dl.acm.org/doi/10.1145/3086512.3086515>
- Fisher, R., Ury, W. L., & Patton, B. (2011). *Getting to yes: Negotiating agreement without giving in* (3rd ed.). Penguin Books.
https://books.google.co.il/books?id=W89fHCJZrcwC&lpg=PR11&ots=a_wmNLbRRs&dq=Getting%20to%20yes%3A%20Negotiating%20agreement%20without%20giving%20n%22%20Fisher%20Ury&lr&pg=PR11#v=onepage&q=Getting%20to%20yes:%20Negotiating%20agreement%20without%20giving%2
- Furth-Matzkin, M. (2017). On the unexpected use of unenforceable contract terms: Evidence from the residential rental market. *Journal of Legal Analysis, 9*(1), 1-49.
<https://doi.org/10.1093/jla/lax002>
- Furth-Matzkin, M. (2019). The harmful effects of unenforceable contract terms: Experimental evidence. *Alabama Law Review, 70*(4), 1031-1086.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3419809
- Garmaise, M. J., & Moskowitz, T. J. (2004). Confronting information asymmetries: Evidence from real estate markets. *The Review of Financial Studies, 17*(2), 405-437.
<https://academic.oup.com/rfs/article-abstract/17/2/405/1576957>
- Govea, J., Ortiz-Gárces, I., Palacios, P., Maldonado Navarro, A., Acurio Del Pino, S., & Villegas-Ch, W. (2026). Technical evaluation of language models adapted for the automation of legal contracts: Clause extraction, classification, and summarization. *Frontiers in Artificial Intelligence, 9*, Article 1782405.
<https://doi.org/10.3389/frai.2026.1782405>
- Kessler, F. (1943). Contracts of adhesion, Some thoughts about freedom of contract. *Columbia Law Review, 43*(5), 629-642. <https://www.jstor.org/stable/1117230>
- Knesset. (n.d.). Rental and Lending Law, 5731-1971. National Legislation Database.
<https://main.knesset.gov.il/apps/legislation/main/laws/2000596>
- LawGeex. (2024). *AI-powered contract review platform for legal teams*.
<https://www.lawgeex.com>
- Levin-Schnur, R., & Procaccia, Y. (2018). More on the Fair Rental Law (Rental and Lending Law (Amendment), 5777-2017). *Mishpatim Online, 12*, 203-212. [Hebrew].
<https://lawjournal.huji.ac.il/article/24/1671>

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33*, 9459-9474.
<https://arxiv.org/abs/2005.11401>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12*, 157-173.
<https://arxiv.org/abs/2307.03172>
- Margalit, T., & Yizhar, D. (2024). Conflicts within the neoliberal state? The limited representation of renters in Israeli housing policies and debates. *Housing Studies*, 1-20. <https://doi.org/10.1080/02673037.2024.2334813>
- Masson, M. E. J., & Waldron, M. A. (1994). Comprehension of legal contracts by non-experts: Effectiveness of plain language redrafting. *Applied Cognitive Psychology*, 8*(1), 67-85. <https://onlinelibrary.wiley.com/doi/abs/10.1002/acp.2350080107>
- Mishly, Y. (2026). *Beyond the shortage: A structural perspective on Israel's housing market*. Shores Institute for Socioeconomic Research.
<https://backend.shores.institute/downloads/research-paper-eng-Mishly-Housing.pdf>
- OECD. (2014). *A revival of the private rental sector of the housing market?* OECD Economics Department Working Papers No. 1170. Paris: OECD Publishing.
<https://doi.org/10.1787/5jxv9f32j0zp-en>
- OpenAI. (n.d.). *text-embedding-3-small* [Model documentation]. OpenAI. Retrieved June 30, 2026, from <https://developers.openai.com/api/docs/models/text-embedding-3-small>
- Rakoff, T. D. (1983). Contracts of adhesion: An essay in reconstruction. *Harvard Law Review*, 96*(6), 1173-1284. <https://www.jstor.org/stable/1341009>
- Rhode, D. L. (2004). *Access to justice*. Oxford University Press.
<https://global.oup.com/academic/product/access-to-justice-9780195143478>
- Shen, Z., Zhang, R., Dell, M., Lee, B. C. G., Carlson, J., & Li, W. (2021). LayoutParser: A unified toolkit for deep learning based document image analysis. *ICDAR 2021*.
<https://arxiv.org/abs/2103.15348>
- Spellbook. (2024). *AI contract drafting and review tool for lawyers*.
<https://www.spellbook.legal>
- Surden, H. (2014). Machine learning and law. *Washington Law Review*, 89*(1), 87-115.
<https://digitalcommons.law.uw.edu/wlr/vol89/iss1/5/>
- Tiersma, P. M. (1999). *Legal language*. University of Chicago Press.
<https://books.google.co.il/books?id=Sq8XXTo3A48C>
- Tsarfaty, R., Seddah, D., Goldberg, Y., Kübler, S., Versley, Y., Candito, M., ... Tounsi, L. (2010). Statistical parsing of morphologically rich languages (SPMRL): What, how and whither. *Proceedings of the NAACL HLT Workshop on SPMRL*.
<https://aclanthology.org/W10-1401/>
- WeCheck. (2024). *Israeli rental background check and tenant verification platform*.
<https://www.wecheck.co.il>